



Al-Kufa University  
Engineering College



Electronic And Communication Department

3<sup>rd</sup> stage

---

## **LABORATORY MANUAL**

---

# **Sequential Logic Circuit Lab**

"Laboratory Experiments of Digital Logic Circuits"

## **Experiment 4**

### **Applications of shift Registers**

### **Case Study 2**

Prepared by:  
Asst.lec. Muhiialdin M.ridha

2023-2024

## 1. Aims:

- Analyze and explain the operation of various circuits that represent an applications of shift registers.

## 2. Theory :

### 2.1 Parallel-to-serial and serial-to-parallel conversion

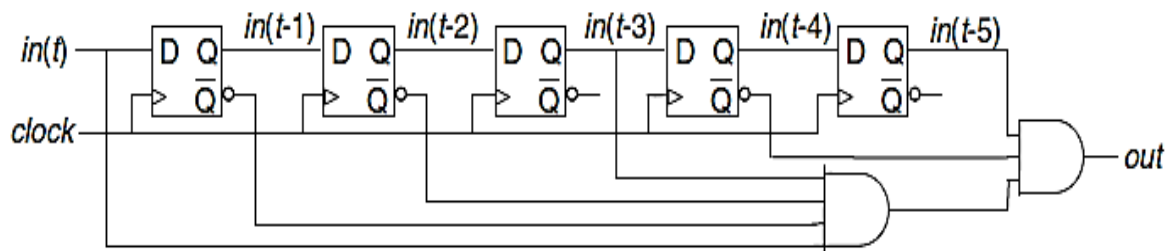
One of the important application of shift registers is the parallel-to-serial and serial-to-parallel conversion in data transmission systems. Assume that a system called “origin” must send n-bit data to another system called “destination” using for that a 1-bit transmission channel as it clear in the figure below. The solution is a parallel-in serial-out shift register on the origin side and a serial-in parallel-out shift register on the destination side. To transmit a data, it is first loaded within register 1 (parallel input); then it is serially shifted out of the register 1 (serial output), it is transmitted on the 1-bit transmission channel, and it is shifted into register 2 (serial input); when all n bits have been transmitted the transmitted data is read from register 2 (parallel output).

- Shift register is used as **Parallel to serial converter**, which converts the parallel data into serial data. It is utilized at the transmitter section after Analog to Digital Converter ADC block.
- Shift register is used as **Serial to parallel converter**, which converts the serial data into parallel data. It is utilized at the receiver section before Digital to Analog Converter DAC block.



## 2.2 Sequence or Pattern Detector

Another application of shift registers is the recognition of sequences. Consider a sequential circuit with a 1-bit input  $in$  and a 1-bit output  $out$ . It receives a continuous string of bits and must generate an output  $out = 1$  every time that the six latest received bits  $in(t)$   $in(t-1)$   $in(t-2)$   $in(t-3)$   $in(t-4)$   $in(t-5)$  are 100101. A solution is shown in the Figure below: a serial-in parallel-out shift register that stores the five values  $in(t-1)$   $in(t-2)$   $in(t-3)$   $in(t-4)$   $in(t-5)$  and generates  $out = 1$  when  $in(t)$   $in(t-1)$   $in(t-2)$   $in(t-3)$   $in(t-4)$   $in(t-5) = 100101$ .



Sequence detector can be designed as a Mealy type or as a Moore type. In a Mealy machine, output depends on the present state and the external input ( $x$ ). Hence, in the diagram, the output is written outside the states, along with inputs.

Sequence detector is of two types:

1. Overlapping
2. Non-Overlapping

In an overlapping sequence detector, the last bit of one sequence becomes the first bit of the next sequence. However, in a non-overlapping sequence detector, the last bit of one sequence does not become the first bit of the next sequence. In this post, we'll discuss the design procedure for non-overlapping 101 Mealy sequence detectors.

For non overlapping case

Input : 0110101011001

Output: 0000100010000

For overlapping case

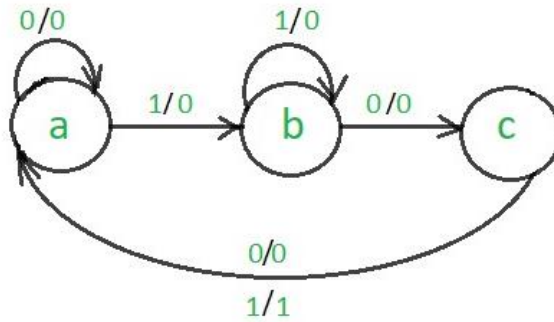
Input : 0110101011001

Output: 0000101010000

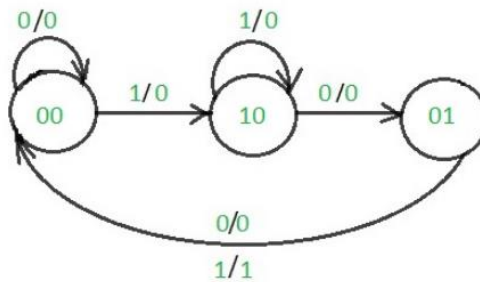
The steps to design a non-overlapping 101 Mealy sequence detectors are:

### Step 1: Develop the state diagram

The state diagram of a Mealy machine for a 101 sequence detector is:



The state diagram after the code assignment is:

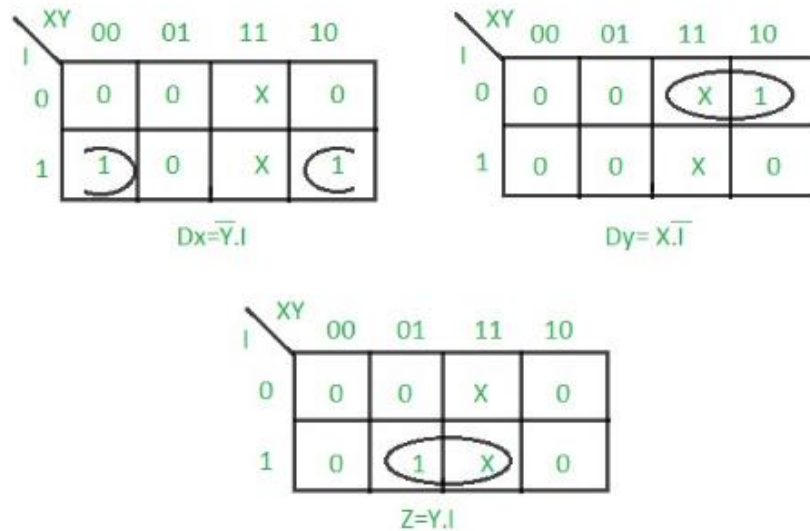


### Step 2: Make Present State/Next State table –

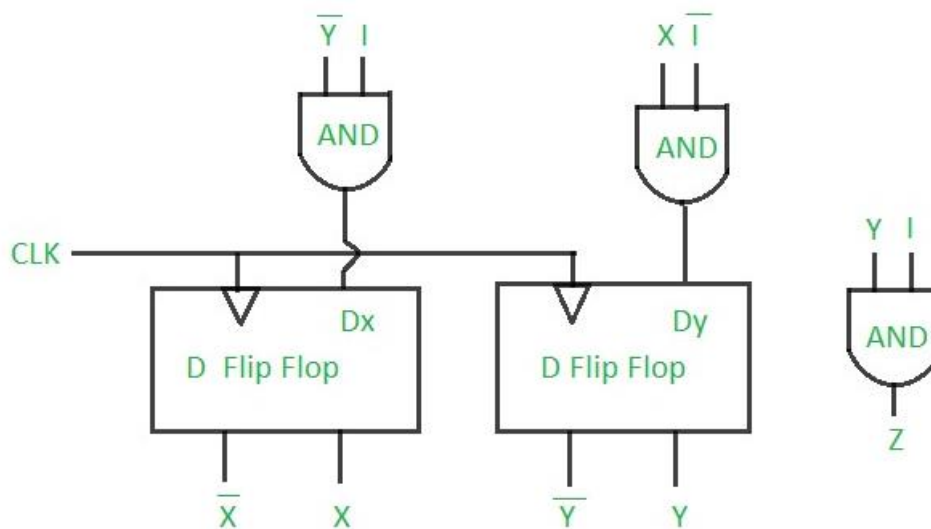
We'll use D-Flip Flops for design purposes.

| Present States |   | i/p | Next States |    | Flip Flop Excitations |    | O/P |
|----------------|---|-----|-------------|----|-----------------------|----|-----|
| X              | Y |     | X'          | Y' | Dx                    | Dy |     |
| 0              | 0 | 0   | 0           | 0  | 0                     | 0  | 0   |
| 0              | 0 | 1   | 1           | 0  | 1                     | 0  | 0   |
| 0              | 1 | 0   | 0           | 0  | 0                     | 0  | 0   |
| 0              | 1 | 1   | 0           | 0  | 0                     | 0  | 1   |
| 1              | 0 | 0   | 0           | 1  | 0                     | 1  | 0   |
| 1              | 0 | 1   | 1           | 0  | 1                     | 0  | 0   |
| 1              | 1 | 0   | X           | X  | X                     | X  | X   |
| 1              | 1 | 1   | X           | X  | X                     | X  | X   |

Step 3: Draw K-maps for Dx, Dy and output (Z)

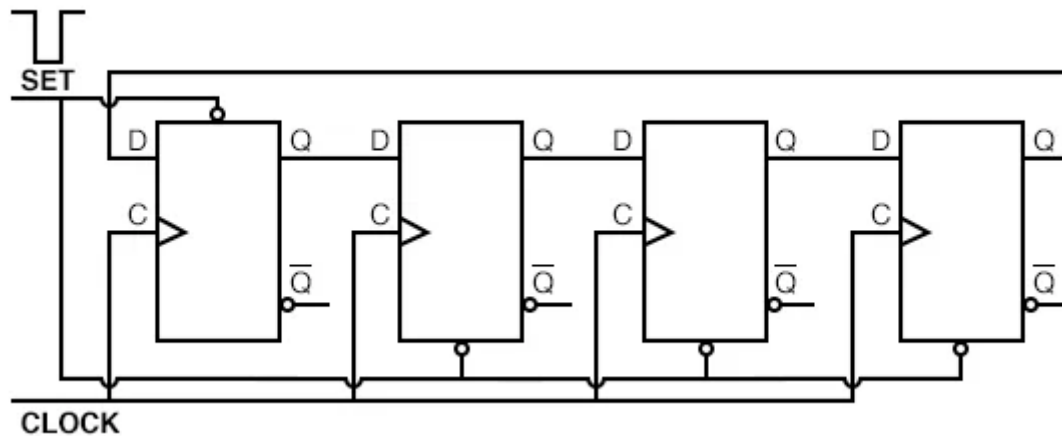


Step 4: Finally implement the circuit



## 2.3 Ring counter

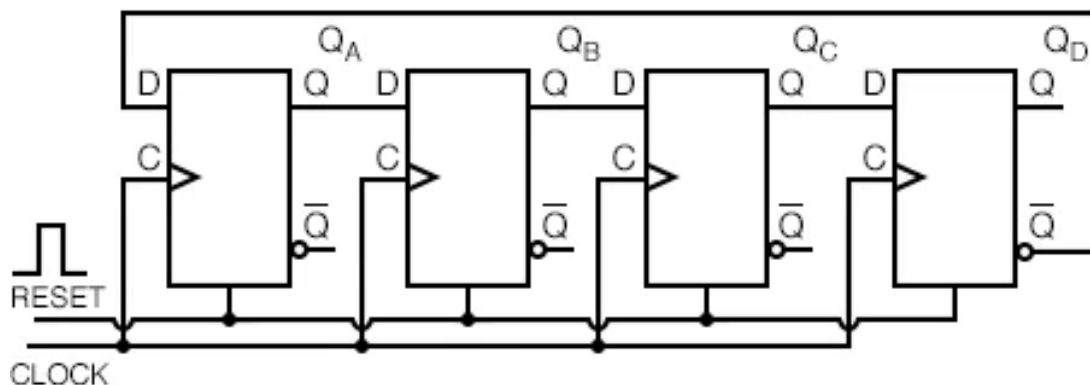
In previous experiment, we discussed the operation of Serial In - Parallel Out SIPO shift register. It accepts the data from outside in serial form and it requires 'N' clock pulses in order to shift 'N' bit data. Similarly, 'N' bit Ring counter performs the similar operation. But, the only difference is that the output of rightmost D flip-flop is given as input of leftmost D flip-flop instead of applying data from outside. A ring counter is a circular shift register with only one flip-flop being set at any particular time; all others are cleared.



- Ring counters can be used to create counters that can count the number of events or cycles in a system. They can be used in various applications, such as timing and synchronization, event counting, and data transmission.
- Ring counters can be used to create digital clocks by counting the number of clock cycles and displaying the time in a suitable format.

## 2.4 Johnson Ring Counter

The **Johnson counter** is similar to the **Ring counter**. The only difference between the **Johnson counter** and the **ring counter** is that the outcome of the last flip flop is passed to the first flip flop as an input. But in **Johnson counter**, the inverted outcome  $Q'$  of the last flip flop is passed as an input. The remaining work of the **Johnson counter** is the same as a **ring counter**. The **Johnson counter** is also referred to as the **Creeping counter**.



## Advantages:

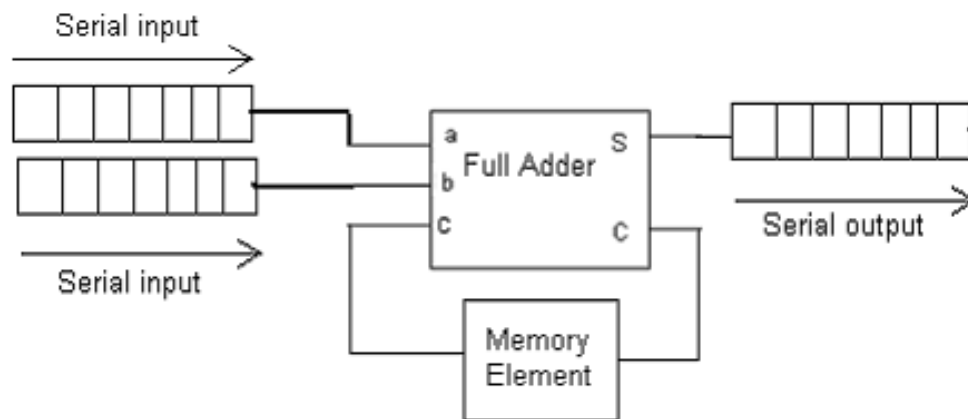
- The number of states is twice the number of flip flops.
- The circuit of the Johnson counter is self-decoding. No need for initialization input.

## 2.5 Serial Adder

Operations in digital computers are usually done in parallel because that is a faster mode of operation. Serial operations are slower because a datapath operation takes several clock cycles, but serial operations have the advantage of requiring fewer hardware components. Initially, register  $A$  and the carry flip-flop are cleared to 0, and then the first number is added from  $B$ . While  $B$  is shifted through the full adder, a second number is transferred to it through its serial input. The second number is then added to the contents of register  $A$ , while a third number is transferred serially into register  $B$ . This can be repeated to perform the addition of two, three, or more four-bit numbers and accumulate their sum in register  $A$ .

The number of full-adder circuits in the parallel adder is equal to the number of bits in the binary numbers, whereas the serial adder requires only one full-adder circuit and a carry flip-flop

| $A$  | $B$  | $S$  | $s_i$ | $c_{i+1}$ |
|------|------|------|-------|-----------|
| 1011 | 0011 | 0000 | 0     | 1         |
| 0101 | 0001 | 1000 | 1     | 1         |
| 0010 | 0000 | 1100 | 1     | 0         |
| 0001 | 0000 | 1110 | 1     | 0         |



### **3. Procedures:**

#### **3.1 Procedure 1:**

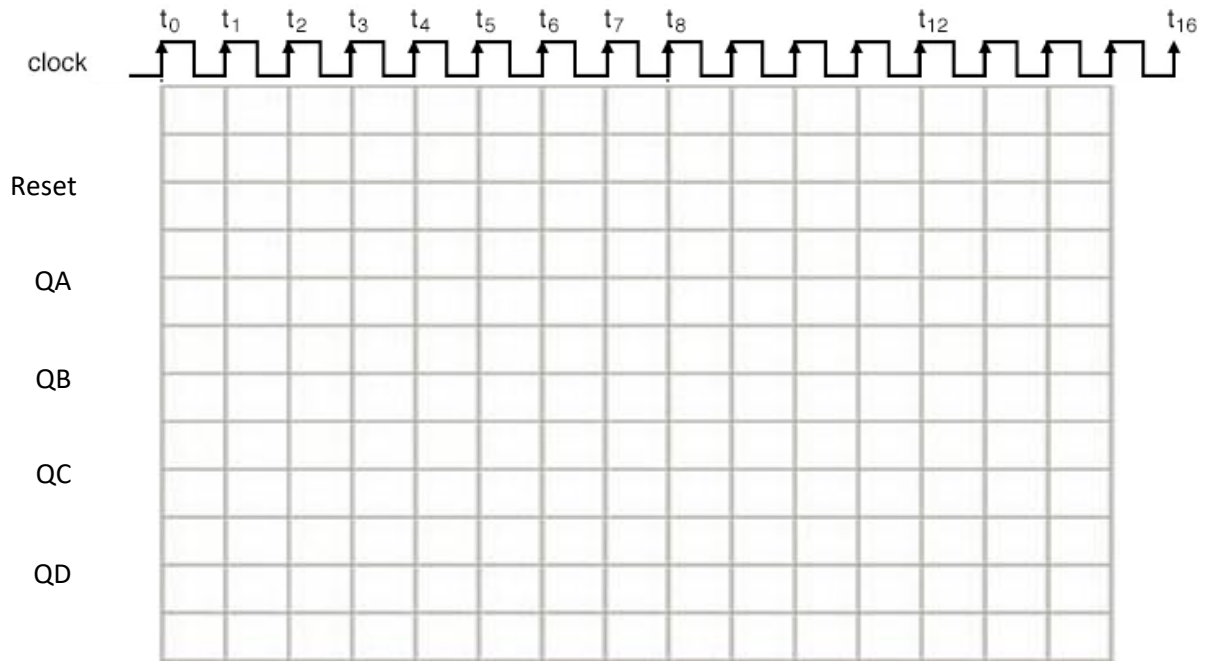
1. Recognize the ring counter circuit shown in section 2.3 then use the lab board to connect it.
2. Draw the output results as a waveforms on the figure below.

|        |  |  |  |  |  |  |  |
|--------|--|--|--|--|--|--|--|
| Clock: |  |  |  |  |  |  |  |
| ORI    |  |  |  |  |  |  |  |
| Q0     |  |  |  |  |  |  |  |
| Q1     |  |  |  |  |  |  |  |
| Q2     |  |  |  |  |  |  |  |
| Q3     |  |  |  |  |  |  |  |

#### **3.2 Procedure 2 :**

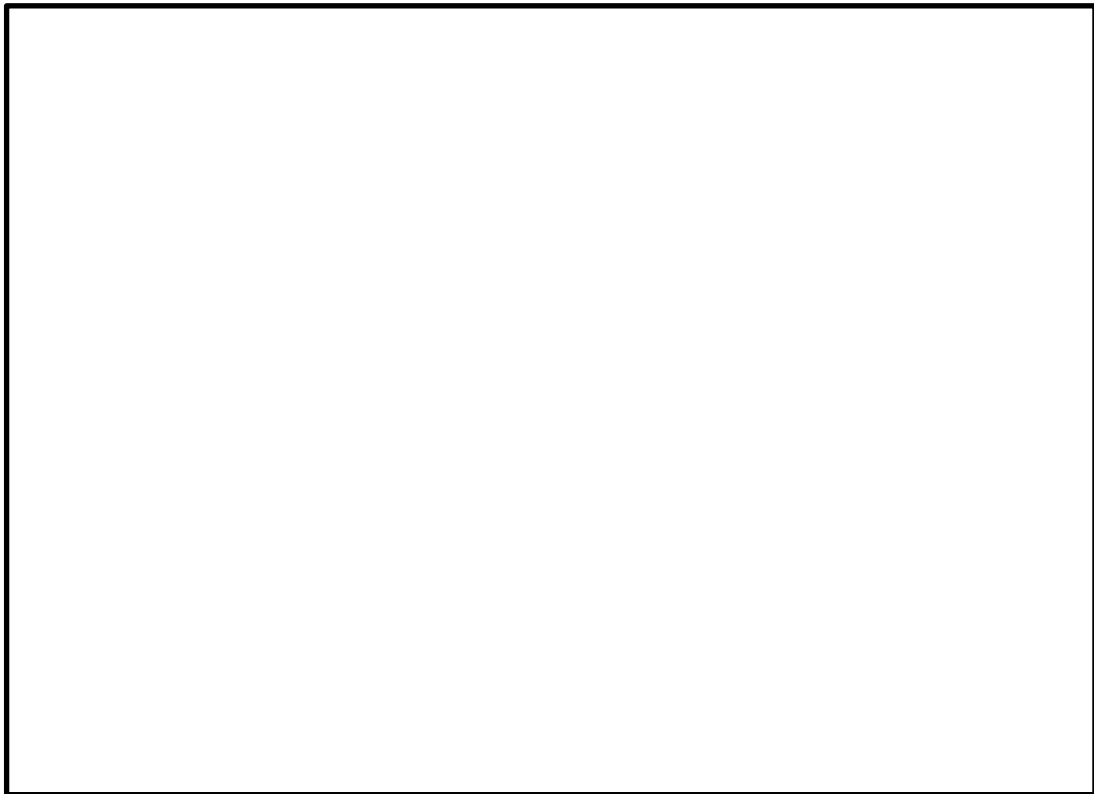
1. Use lab board to connect the circuit shown in section 3.4
2. Draw the output results as a waveforms on the figure below.





### 3.3 Procedure 3 :

1. For sequence detector in section 2.2, design a circuit that can detect input binary sequence = 0111.
2. Draw the state diagram of the Mealy machine for a 0111 sequence detector.



3. From the state diagram fill the state table below.

| Present state |       | OB | Next state |        | Flip flop inputs |       | Output |
|---------------|-------|----|------------|--------|------------------|-------|--------|
| $q_0$         | $q_1$ |    | $q_0'$     | $q_1'$ | $D_0$            | $D_1$ | Y      |
| 0             | 0     | 0  |            |        |                  |       |        |
| 0             | 0     | 1  |            |        |                  |       |        |
| 0             | 1     | 0  |            |        |                  |       |        |
| 0             | 1     | 1  |            |        |                  |       |        |
| 1             | 0     | 0  |            |        |                  |       |        |
| 1             | 0     | 1  |            |        |                  |       |        |
| 1             | 1     | 0  |            |        |                  |       |        |
| 1             | 1     | 1  |            |        |                  |       |        |

4. Use k-map to find the equation for:

$D_0 =$

$D_1 =$

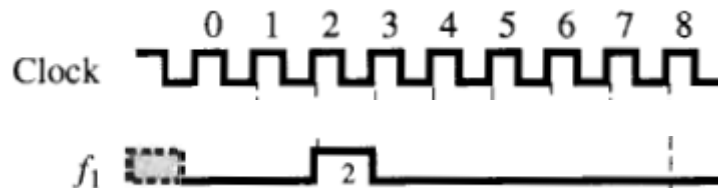
$Y =$

5. From the equations in step 4 draw then connect the circuit diagram

## 4. Discussions

1. For the Johnson counter in procedure2 adjust the circuit by adding any combinational circuit you need to get a control signals f1 and f2.

- The first timing control signal f1 will generate a pulse that will go high on the leading edge of the second clock pulse and go low at the next clock cycle. Look at the figure below.



- The second timing control signal f2 will generate a pulse that will go high on the leading edge of the fifth clock pulse and go low at the next clock cycle.

## 5. Resources:

- <https://www.geeksforgeeks.org/design-101-sequence-detector-mealy-machine/>
- <https://www.youtube.com/watch?v=XNAKL7NIOM&list=PLBlnK6fEyqRjMH3mWf6kwqiTbT798eAOm&index=172>
- <https://www.youtube.com/watch?v=OVTjSu1jK48>

### SN7400 SERIES SHIFT REGISTERS

| Device | Features                                                                                                 |
|--------|----------------------------------------------------------------------------------------------------------|
| 7491A  | 8-Bit, serial in, serial out                                                                             |
| 7496   | 5-Bit, serial in, serial out, asynchronous preset, parallel out, common clear                            |
| 74164  | 8-Bit, serial in, serial out or parallel out, common clear                                               |
| 74165  | 8-Bit, serial in, serial out, asynchronous load, clock inhibit                                           |
| 74179  | 4-Bit, serial in, serial out, common clear, synchronous load, parallel out, synchronous data hold        |
| 74194  | 4-Bit, bidirectional, serial in, serial out, synchronous load, parallel out, clock inhibit, common clear |